Course Project Report: Smart Campus Management System (SCMS)

1. Title Page

- **Course:** CS-202: Object-Oriented Programming (OOP)
- **Course Instructor:** Lab Instructor, HITEC University Taxila
- **Project Title:** Smart Campus Management System (SCMS)
- **Student Name:** Syed Muhammad Kashif
- **Roll Number:** 25-Cs-016
- **Date:** June 18, 2026
- **Department:** Department of Computer Science
- **Institution:** HITEC University Taxila
- **GitHub Repository:** https://github.com/25-Cs-016/HITEC-OOP-SCMS-25-Cs-016

2. Introduction

Problem Statement

Managing a modern university campus involves coordinating multiple complex domains, including person registries (students, graduate students, faculty, and staff), course scheduling and enrollments, library catalog management, room allotments, and financial auditing. Traditional manual operations or fragmented databases result in data inconsistency, resource wastage, and processing delays.

System Goals

The Smart Campus Management System (SCMS) aims to resolve these challenges by providing a consolidated, object-oriented desktop administration suite. The key goals

include:

1. **Unified Registries:** Maintaining records of all campus personnel under a polymorphic hierarchy.
2. **Encapsulated Enrollments:** Managing course lists, registering students, and handling waitlists dynamically.
3. **Automated Library:** Facilitating generic book/journal catalog indexing, fstream-based persistence, and automatic fine calculations for late returns.
4. **Optimized Hostels:** Assigning students to rooms in hostel blocks while avoiding multiple-inheritance diamond collisions.
5. **Secure Finance:** Managing fee payments and generating unique auto-incrementing invoices.

---

3. System Design & UML Class Diagram

UML Class Diagram

The software architecture is visualized in `docs/class_diagram.png`. It represents a clean modularized design.

Explanation of Relationships

- **Inheritance:** `Student`, `Faculty`, and `Staff` inherit from the abstract base class `Person`. `GradStudent` inherits from `Student` in a multi-level fashion.
- **Multiple & Virtual Inheritance:** `HostelManager` inherits from both `Accommodation` and `Reportable` interfaces, which in turn inherit virtually from `HostelService` to resolve the diamond problem.
- **Composition:** `HostelManager` is composed of `HostelBlock` objects, which are composed of `Room` objects (lifecycle dependencies are managed by the parent class).
- **Aggregation:** `Course` references a `Faculty` instructor (the instructor continues to exist even if the course is deleted).
- **Association:** `Enrollment` links `Student` and `Course` to document specific registration details.

---

4. OOP Concepts Implementation

Below is the verification of how and where each of the 25 required OOP concepts is implemented:

1. Classes & Objects

Implemented across all files. Classes model real-world entities, and objects are instantiated in `main.cpp`.

Snippet:

```
Course oopCourse("CS-202", "Object-Oriented Programming", 4, facultyPtr, 30);
```

2. Encapsulation (Getters/Setters)

Data members are made `private` or `protected` and accessed via public methods.

Snippet (Person.h):

```
class Person {
protected:
    std::string name;
public:
    std::string getName() const { return name; }
};
```

3. Constructors (Default, Param, Copy)

Custom constructors implemented in `Person`, `Course`, and `FeeRecord`.

Snippet (FeeRecord.cpp):

```
FeeRecord::FeeRecord(std::shared_ptr<Student> student, double semFee, double ho
    : studentRef(student), semesterFee(semFee), hostelFee(hostelFee), libraryFi
```

4. Destructors

Custom destructors ensure proper cleanup and logging.

Snippet (HostelManager.cpp):

```
HostelManager::~HostelManager() {
    std::cout << "[HostelManager Destructor] Deallocating hostel blocks...\n";
}
```

5. Single Inheritance

`Student` inherits directly from `Person`.

Snippet (Student.h):

```
class Student : public Person { ... };
```

6. Multi-level Inheritance

`GradStudent` inherits from `Student`, which inherits from `Person`.

Snippet (GradStudent.h):

```
class GradStudent : public Student { ... };
```

7. Multiple Inheritance

`HostelManager` inherits from both `Accommodation` and `Reportable`.

Snippet (HostelManager.h):

```
class HostelManager : public Accommodation, public Reportable { ... };
```

8. Virtual Inheritance

Resolves the diamond problem by virtually inheriting the common top-base class `HostelService`.

Snippet (HostelManager.h):

```
class Accommodation : virtual public HostelService { ... };
class Reportable : virtual public HostelService { ... };
```

9. Abstract Classes & Pure Virtual Functions

`Person`, `LibraryItem`, and `Accommodation` declare pure virtual methods making them abstract.

Snippet (Person.h):

```
virtual void displayInfo() const = 0;
```

10. Runtime Polymorphism

Calling overridden methods dynamically via base class smart pointers.

Snippet (main.cpp):

```
std::shared_ptr<Person> p = studentPtr;
p->displayInfo(); // Dynamically dispatches to Student::displayInfo()
```

11. Operator Overloading (==, <<, -=)

Overloaded `==` in `Course`, `<<` in `Course` / `Invoice`, and `-=` in `FeeRecord`.

Snippet (FeeRecord.cpp):

```
FeeRecord& FeeRecord::operator-=(double paymentAmount) {
    totalPaid += paymentAmount;
    balance -= paymentAmount;
    return *this;
}
```

12. Friend Functions

Overloaded stream insertions are declared as friends to inspect private class fields.

Snippet (Invoice.h):

```
friend std::ostream& operator<<(std::ostream& os, const Invoice& invoice);
```

13. Static Members

`invoiceCounter` tracks and auto-increments IDs.

Snippet (Invoice.cpp):

```
int Invoice::invoiceCounter = 1000;
```

14. Copy Constructor (Deep Copy)

`FeeRecord` implements a deep copy constructor that duplicates the underlying student object.

Snippet (FeeRecord.cpp):

```
FeeRecord::FeeRecord(const FeeRecord& other) {
    if (other.studentRef) {
        this->studentRef = std::make_shared<Student>(*other.studentRef);
    }
    // ...
}
```

15. Move Semantics (Rule of Five)

`Invoice` implements custom move constructor and move assignment operator.

Snippet (Invoice.cpp):

```
Invoice::Invoice(Invoice&& other) noexcept
    : invoiceID(other.invoiceID), date(std::move(other.date)), items(std::move(
```

16. Function Templates

Templated title searching that is generic across books and journals.

Snippet (Library.h):

```
template <typename ItemType>
ItemType* searchByTitle(const std::string& title);
```

17. Class Templates

`Library<T>` is defined generically to handle database management for different library items.

Snippet (Library.h):

```
template <typename T>
class Library { ... };
```

18. STL Containers

Extensively utilizes `std::vector` for registries and `std::map` for checkout logs.

Snippet (main.cpp):

```
std::map<std::string, std::vector<std::pair<std::string, std::string>>> issuedI
```

19. Exception Handling

Throws custom exceptions when capacity is exceeded or items are overdue.

Snippet (Course.cpp):

```
if (enrolledStudents.size() >= maxCapacity) {
    throw SCMS::CapacityExceededException(courseCode, maxCapacity);
}
```

20. File I/O (fstream)

Saves and loads library records using `std::ifstream` and `std::ofstream`.

Snippet (Library.h):

```
std::ofstream outFile(filepath);
// write fields to file...
```

21. Namespaces

`SCMS::Reports` and `SCMS::Utils` encapsulate application helper logic and print templates.

Snippet (Reports.h):

```
namespace SCMS { namespace Utils { ... } }
```

22. Smart Pointers

Used `std::shared_ptr` for shared objects like `Student` and `std::unique_ptr` for `HostelManager`.

Snippet (main.cpp):

```
std::unique_ptr<HostelManager> hostelSubsystem;
```

23. Lambda Expressions

Used for sorting and custom predicate lookups in algorithms.

Snippet (Reports.cpp):

```
std::sort(students.begin(), students.end(), [](const std::shared_ptr<Student>&
    return a->getGPA() > b->getGPA();
});
```

24. Composition

`HostelBlock` is composed inside `HostelManager`. If the manager is destroyed, the blocks are destroyed.

Snippet (HostelManager.h):

```
std::vector<HostelBlock> blocks;
```

25. Aggregation

`Course` contains a reference/pointer to `Faculty` without managing its lifetime.

Snippet (Course.h):

```
std::shared_ptr<Faculty> instructor;
```

---

5. Module Descriptions

1. **Person Hierarchy:** Represents the core records database using polymorphism. Derived objects are stored in a unified registry, demonstrating access specifiers.
2. **Course & Enrollment Management:** Facilitates course registrations, checks course limits, and uses waitlists.
3. **Library System:** Manages catalogs. Demonstrates templates and fstream-based persistence.
4. **Fee & Finance Management:** Handles financial accounts. Validates Rule of Three/Five.
5. **Hostel Management:** Resolves hostel assignments and implements multiple inheritance, avoiding multiple-inheritance diamond collisions.
6. **Reporting & Utilities:** Uses lambdas to sort and filter campus states, producing a consolidated text report.

---

6. GitHub Workflow & Branching Strategy

Our development was structured around clear git workflows:

- **Branching Strategy:** Main branch used as production, with individual modules developed on feature branches (e.g. `feature/person-hierarchy`, `feature/library-system`) and merged via Pull Requests.

- **Commit Messages:** Followed a strict format: `[Module] Description`. For example: `[Person] Implement abstract base class Person`.

---

7. Challenges & Solutions

1. **Circular Inclusion:** Resolved circular headers between `Student` and `Course` by storing enrolled courses as simple string codes in `Student` and resolving them dynamically.
2. **Diamond Problem in Hostel Subsystem:** Resolved multiple-inheritance base conflicts by inheriting `HostelService` virtually in both `Accommodation` and `Reportable` classes, initializing it explicitly in `HostelManager`.
3. **Type Slicing in Templates:** Solved type slicing in generic library listings by specializing `Library<T>` for specific types `Book` and `Journal`.

---

8. Verification & Program Output

Compilation

The program compiles warning-free with `g++ -std=c++17 -Wall -Wextra`:

```
g++ -std=c++17 -Wall -Wextra src/person/*.cpp src/course/*.cpp src/library/*.cp
```

Smoke Test Output

The execution displays the central initializer loading 5 students from `students.csv`, loading the library catalogs, setting up instructors, and starting the interactive main menu.

---

9. Conclusion

This project demonstrated the benefits of object-oriented design in managing large-scale application domains. Through encapsulation, polymorphism, templates, smart pointers,

and exception safety, we built a stable, memory-efficient administration suite.

10. References

- *C++ How to Program (10th Edition)* by Deitel & Deitel
- *Effective Modern C++* by Scott Meyers
- cppreference.com